POLITECHNIKA GDAŃSKA
WYDZIAŁ FIZYKI TECHNICZNEJ I MATEMATYKI STOSOWANEJ

Imię i nazwisko studenta: Natalia Właśniak
Nr albumu: 161316
Studia pierwszego stopnia
Forma studiów: stacjonarne
Kierunek studiów: Matematyka
Specjalność: Matematyka stosowana

PRACA DYPLOMOWA LICENCJACKA

Tytuł pracy w języku polskim: Algorytmy dotyczące liczb pierwszych.

Tytuł pracy w języku angielskim: Prime numbers algorithms.

Potwierdzenie przyjęcia pracy	
Opiekun pracy *podpis*	Kierownik Katedry/Zakładu (pozostawić właściwe) *podpis*
dr inż. Magdalena Lemańska	

Data oddania pracy do dziekanatu:

POLITECHNIKA GDAŃSKA
WYDZIAŁ FIZYKI TECHNICZNEJ I MATEMATYKI STOSOWANEJ

OŚWIADCZENIE dotyczące pracy dyplomowej zatytułowanej: Algorytmy dotyczące liczb pierwszych.

Imię i nazwisko studenta: Natalia Właśniak
Data i miejsce urodzenia: 12.12.1995, Bydgoszcz
Nr albumu: 161316
Wydział: Wydział Fizyki Technicznej i Matematyki Stosowanej
Kierunek: matematyka
Poziom kształcenia: pierwszy
Forma studiów: stacjonarne

Świadomy(a) odpowiedzialności karnej z tytułu naruszenia przepisów ustawy z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz.U. z 2017 poz. 880 z późn. zm.) i konsekwencji dyscyplinarnych określonych w ustawie Prawo o szkolnictwie wyższym (Dz.U. 2017 poz. 2183 z późn. zm.),[1] a także odpowiedzialności cywilnoprawnej oświadczam, że przedkładana praca dyplomowa została opracowana przeze mnie samodzielnie.

Niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadaniem tytułu zawodowego.

Wszystkie informacje umieszczone w ww. pracy dyplomowej, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami zgodnie z art. 34 ustawy o prawie autorskim i prawach pokrewnych.

Potwierdzam zgodność niniejszej wersji pracy dyplomowej z załączoną wersją elektroniczną.

Gdańsk, dnia ....
podpis studenta

[1] Ustawa z dnia 27 lipca 2005 r. Prawo o szkolnictwie wyższym:
Art. 214 ustęp 4. W razie podejrzenia popełnienia przez studenta czynu podlegającego na przypisaniu sobie autorstwa istotnego fragmentu lub innych elementów cudzego utworu rektor niezwłocznie poleca przeprowadzenie postępowania wyjaśniającego.
Art. 214 ustęp 6. Jeżeli w wyniku postępowania wyjaśniającego zebrany materiał potwierdza popełnienie czynu, o którym mowa w ust. 4, rektor wstrzymuje postępowanie o nadanie tytułu zawodowego do czasu wydania orzeczenia przez komisję dyscyplinarną oraz składa zawiadomienie o popełnieniu przestępstwa.

Streszczenie

Tematem pracy są algorytmy dotyczące liczb pierwszych. Podzielono ją na dwie części ze względu na typ omawianych w nich algorytmów. W pierwszej części w głównej mierze skupiono się na omówieniu dwóch probabilistycznych testów pierwszości - testu Fermata i testu Millera-Rabina. W drugiej części przedstawiono trzy algorytmy generujące wszystkie liczby pierwsze aż do wybranej liczby - sito Eratostenesa, sito Sundarama i sito Atkina - a następnie w oparciu o napisany kod w języku Python porównano je ze sobą pod względem czasu działania w zależności od zadanego przedziału.

Słowa kluczowe: algorytmy, liczby pierwsze, sita, testy pierwszości.

Abstract

This work is devoted to prime numbers algorithms. It is divided into two parts in view of the type of algorithms discussed in them. The first part focuses mainly on the discussion of two probabilistic primality tests - the Fermat test and the Miller-Rabin test. In the second part three algorithms that generate all prime numbers up to a desired limit are presented - Eratosthenes sieve, Sundaram sieve and Atkin sieve - and then based on the code written in Python they are compared with each other in terms of operating time depending on the given limit of the interval.

Key words: algorithms, primality tests, prime numbers, sieves.

Spis treści

Wstęp i cel pracy **6**

1 Testowanie pierwszości **7**
1.1 Algorytmy deterministyczne 7
1.2 Algorytmy probabilistyczne 8
1.2.1 Test Fermata 8
1.2.2 Test Millera-Rabina 11

2 Generowanie liczb pierwszych **18**
2.1 Sito Eratostenesa 18
2.2 Sito Sundarama 20
2.3 Sito Atkina 21
2.4 Porównanie przykładowo zaimplementowanych sit. Wybranie najszybszego z nich w zależności od długości zadanego przedziału 24

Podsumowanie **29**

Wykaz literatury **30**

Dodatek A **31**

Wstęp i cel pracy

Celem niniejszej pracy jest przedstawienie wybranych algorytmów wraz z pełnym bądź częściowym uzasadnieniem poprawności ich działania oraz zilustrowanie ich przykładami. Większa część pracy opiera się na działaniach z arytmetyki modularnej.

Pierwszy rozdział poświęcono testom pierwszości, a w szczególności probabilistycznym testom pierwszości - teście Fermata oraz teście Millera-Rabina. Oba testy się dopełniają, jako że opis pierwszego z nich jest tak naprawdę wstępem do zrozumienia działania drugiego.

W drugim rozdziale przedstawiono sita generujące liczby pierwsze - sito Eratostenesa, sito Sundarama oraz sito Atkina - każde z nich działa w inny sposób (nie dopełniają się jak wymienione wyżej testy pierwszości), stąd został również zrealizowany pomysł porównania ich implementacji. Ma ono na celu pokazanie, jak mogą zachowywać się sita dla pewnych implemetacji, ale nie jest ono wzorcem dla ogółu wdrożeń algorytmów, jako że mogą one zawsze zostać lepiej zoptymalizowane. Do pracy został dołączony napisany kod programu w języku Python, który był pomocny podczas analizy zestawienia sit.

Wybór algorytmu dowolnego typu zależy od tego, gdzie ma on znaleźć swoje zastosowanie, zatem od parametrów, które są najistotniejsze dla osoby przeprowadzającej dany algorytm, np. poprawność, złożoność obliczeniowa.

Rozdział 1

Testowanie pierwszości

Chcąc się dowiedzieć, czy dana liczba jest pierwsza, korzysta się z testów pierwszości. Można je przeprowadzić, wykorzystując zarówno algorytmy deterministyczne jak i probabilistyczne. Poniżej przedstawiono jedynie najprostszy z deterministycznych testów pierwszości, ponieważ ze względu na szersze zastosowanie probabilistycznych testów pierwszości to właśnie im w głównej mierze został poświęcony ten rozdział.

1.1 Algorytmy deterministyczne

Deterministyczne testy pierwszości to algorytmy, które, mając zadaną konkretną liczbę n jako daną wejściową, po wielokrotnym wykonaniu kolejno tych samych procedur zwracają zawsze tę samą daną wyjściową, tj. „n jest liczbą pierwszą" albo „n jest liczbą złożoną". Poniżej przedstawiono najprostszy z nich.

Algorytm 1 Test naiwny

- Podajemy liczbę naturalną $n > 2$, której pierwszość chcemy sprawdzić.
- Dzielimy n przez kolejne liczby $2, 3, ..., \lfloor\sqrt{n}\rfloor$. Jeśli żadna z nich nie dzieli n bez reszty, to n jest pierwsza. W przeciwnym razie jest złożona.

Dzielenie n jedynie przez liczby naturalne mniejsze lub równe $\lfloor\sqrt{n}\rfloor$ wynika z poniższego twierdzenia.

Twierdzenie 1.1 ([6]). *Jeśli n jest liczbą złożoną, to n posiada dzielnik pierwszy p taki, że $p \leqslant \sqrt{n}$.*

Dowód. Niech p będzie najmniejszym dzielnikiem pierwszym liczby n. Skoro $n = rs$, to $p \leqslant r$ oraz $p \leqslant s$. Stąd $p^2 \leqslant rs = n$, zatem $p \leqslant \sqrt{n}$. □

Powyższy algorytm można zoptymalizować, dzieląc n jedynie przez kolejne liczby pierwsze $p < \sqrt{n}$, które można wygenerować m.in. metodą sita Eratostenesa, która została przedstawiona w rozdziale 2.

1.2 Algorytmy probabilistyczne

Probabilistyczne testy pierwszości to algorytmy, których informacja zwrotna na temat pierwszości/złożoności liczby n zależy od pewnej wielokrotnie losowo wybranej liczby a. Otóż po kilkukrotnym przeprowadzeniu takiego testu dla tej samej liczby n w zależności od losowanych wartości a informacja zwrotna może być postaci „n jest złożona" albo „n jest pierwsza" (co z powodu elementu losowości w algorytmach probabilistycznych tak naprawdę oznacza tylko tyle, że n jest prawdopodobnie pierwsza). Tego typu algorytmy nigdy nie uznają liczby pierwszej za złożoną, ale może się zdarzyć sytuacja, w której liczba złożona zostanie uznana za pierwszą.

1.2.1 Test Fermata

Test pierwszości Fermata to algorytm probabilistyczny, który opiera się na małym twierdzeniu Fermata. Jest on mało skomplikowany, ale za to niepewny (w porównaniu np. do testu Millera-Rabina) ze względu na istnienie liczb pseudopierwszych, a w szczególności liczb Carmichaela, które pomimo tego, że są liczbami złożonymi, bywają przez test Fermata uznawane za pierwsze. Więcej informacji na ich temat znajduje się w dalszej części podrozdziału. Najpierw zostaną przedstawione małe twierdzenie Fermata oraz sam algorytm.

Twierdzenie 1.2 (**Małe twierdzenie Fermata** [4]). *Jeżeli p jest liczbą pierwszą, zaś a liczbą całkowitą, to*

$$a^p \equiv a \ (mod\ p) \ . \tag{1.1}$$

Dowód. Niech $a \in \mathbb{N}_+$ oraz p jest liczbą pierwszą. Pokażemy, że zachodzi wzór (1.1).
Krok początkowy dla $a = 1$:

$$L = 1^p \equiv 1 \ (mod\ p) = P \ .$$

Krok indukcyjny:
Zakładamy, że dla pewnego $a \in \mathbb{N}_+$ zachodzi $a^p \equiv a \pmod{p}$. Udowodnimy prawdziwość kongruencji (1.1) dla $a + 1$, czyli

$$(a + 1)^p \equiv (a + 1) \ (mod\ p) \ .$$

Dowód indukcyjny:
Korzystając ze wzoru na szereg Newtona, otrzymujemy:

$$(a+1)^p = \sum_{k=0}^{p} \binom{p}{k} a^k = a^p + a^0 + \sum_{k=1}^{p-1} \binom{p}{k} a^k \ .$$

$$\binom{p}{k} = \frac{p!}{k!(p-k)!} = \frac{p(p-1)(p-2)...(p-k+1)}{k!}$$

Dla $0 < k < p$ żaden z czynników $k!$ nie przyjmuje wartości p, więc cały symbol Newtona jest wielokrotnością p. Stąd $\binom{p}{k} \equiv 0 \pmod{p}$. Zatem

$$(a+1)^p \equiv a^p + a^0 + \sum_{k=1}^{p-1} \binom{p}{k} a^k \equiv a^p + a^0 \overset{\text{zał. ind.}}{\equiv} a + 1 \ (mod\ p) \ .$$

Stąd i z zasady indukcji matematycznej wzór (1.1) jest prawdziwy dla każdej liczby naturalnej $a \geqslant 1$.

□

Wniosek 1.3 ([5]). *Jeżeli p jest liczbą pierwszą, to dla każdego a względnie pierwszego z p mamy*

$$a^{p-1} \equiv 1 \ (mod\ p) \ . \tag{1.2}$$

Uwaga 1.4. *Stwierdzenie odwrotne do wniosku 1.3 nie jest w ogólności prawdziwe.*

Dowód. Aby to pokazać, wystarczy znaleźć liczbę a względnie pierwszą z liczbą złożoną n, taką że n spełnia równość (1.2). Weźmy liczby $n = 341 = 11 \cdot 31$ i $a = 2$, które są względnie pierwsze. Podstawmy do wzoru (1.2) nasze a i n:

$$a^{n-1} = 2^{341-1} \equiv 1 \ (mod\ 341) \ .$$

Kongruencja (1.2) jest spełniona. Zatem pokazaliśmy, że z tego, że n spełnia równość (1.2), nie wynika, że n jest zawsze pierwsza.

□

Definicja 1.5 ([5]). *W teście Fermata taką liczbę a, dla której $NWD(a, n) = 1$ oraz $a^{n-1} \not\equiv 1$ (mod n) będziemy nazywać świadkiem złożoności dla n, ponieważ zaświadcza ona, że n jest złożona.*

W poniższym algorytmie k oznacza liczbę powtórzeń głównej jego części i im większa zostanie podana jej wartość, tym większa będzie szansa na znalezienie potencjalnego świadka złożoności dla n za cenę wydłużenia jego pracy.

Algorytm 2 Test Fermata

- Podajemy liczbę $n > 2$, której pierwszość chcemy sprawdzić.
- Podajemy liczbę k oznaczającą maksymalną ilość powtórzeń kolejnych kroków.
- Powtarzamy poniższe procedury k razy, o ile algorytm wcześniej nie stwierdzi, że n jest złożona.
  - — Losujemy liczbę całkowitą $1 < a < n$.
  - — Sprawdzamy, czy $NWD(a, n) = 1$. Jeśli $NWD(n, a) > 1$, to n jest złożona i można zakończyć algorytm.
  - — Obliczamy a^{n-1} mod n. Jeżeli

    $$a^{n-1} \not\equiv 1 \ (mod\ n) \ ,$$

    to n jest złożona i można zakończyć algorytm.
- Jeżeli po k powtórzeniach nie wykryto świadka złożoności dla n (czyli dla każdej z k wylosowanych liczb a spełniona została równość $a^{n-1} \equiv 1 \pmod{n}$), to algorytm orzeka, że n jest pierwsza.

Poniżej przedstawiono przykład przebiegu testu Fermata dla liczby 197.

Przykład 1.6. *Prześledźmy algorytm Fermata dla $n = 197$ w maksymalnie dwóch powtórzeniach. Rozpatrzmy pierwszą iterację ($k = 1$): losujemy liczbę $1 < a < 197$. Wylosowano $a = 35$, $NWD(35, 197) = 1$. Sprawdzamy, czy spełniona jest równość (1.2):*

$$a^{n-1} = 35^{197-1} \equiv 1 \ (mod\ 197) \ .$$

Kongruencja jest spełniona, czyli po pierwszej iteracji nie znaleziono świadka złożoności dla 197. Zatem przechodzimy do drugiej iteracji ($k = 2$): losujemy liczbę $a = 160$, $NWD(160, 197) = 1$. Otrzymujemy, że

$$a^{n-1} = 160^{197-1} \equiv 1 \ (mod\ 197) \ .$$

Dla tak zadanych liczb kongruencja jest ponownie spełniona, zatem algorytm zwróciłby informację, że 197 jest pierwsza.

W algorytmie Fermata jeśli n jest liczbą pierwszą, to spełnia równość (1.2) dla każdego $a \in \mathbb{Z}_n^* = \{k \in \mathbb{Z}_n : NWD(k, n) = 1\}$ (co wynika z wniosku 1.3), stąd algorytm zawsze zwróci dla niej poprawną informację, że jest pierwsza (tak jak w przykładzie 1.6). Zatem jeśli znajdziemy jakiekolwiek $a \in \mathbb{Z}_n^*$ takie, że n nie spełnia równości (1.2), to n na pewno jest liczbą złożoną.

W przypadku gdy zostanie wylosowana liczba $a \in \mathbb{Z}_n^*$ i nie będzie ona świadkiem złożoności dla n, algorytm może się pomylić, orzekając o liczbie złożonej n, że jest pierwsza. Element prawdopodobieństwa w teście Fermata wynika z istnienia liczb pseudopierwszych przy podstawie a.

Definicja 1.7 ([2]). *Mówimy, że liczba n jest pseudopierwsza przy podstawie a, jeśli n jest liczbą złożoną oraz $a^{n-1} \equiv 1 \ (mod\ n)$.*

Przykładem takiej liczby jest liczba złożona $n = 341 = 11 \cdot 31$, która jest pseudopierwsza przy podstawie 2. Po przeprowadzeniu testu Fermata jednokrotnie dla wylosowanego $a = 2$ otrzymano by informację, że liczba 341 jest pierwsza, co jest błędnym określeniem. W tym przypadku kolejne przeprowadznie algorytmu dla innej wylosowanej liczby, np. $a = 3$ zwróciłoby poprawną informację (co zostało przedstawione w przykładzie 1.8).

Przykład 1.8. *Chcemy spawdzić pierwszość liczby $n = 341$. Niech $k = 3$. Spójrzmy na pierwszą iterację ($k = 1$): losujemy liczbę $1 < a < 341$. Wylosowano $a = 2$, $NWD(2, 341) = 1$. Sprawdzamy, czy spełniona jest równość (1.2):*

$$a^{n-1} = 2^{341-1} \equiv 1 \ (mod\ 341) \ .$$

Równość jest spełniona, zatem przechodzimy do drugiej iteracji ($k = 2$): losujemy liczbę $a = 3$, $NWD(3, 341) = 1$. Otrzymujemy, że

$$a^{n-1} = 3^{341-1} \equiv 56 \not\equiv 1 \ (mod\ 341) \ ,$$

więc znaleźliśmy liczbę $a = 3$, która jest świadkiem złożoności dla 341, czyli 341 jest złożona (koniec algorytmu).

Okazuje się, że nie udałoby się zawsze dla każdej liczby n pseudopierwszej wyeliminować błędu określenia pierwszości, sprawdzając, czy równanie (1.2) jest spełnione dla innej podstawy a, ze względu na istnienie liczb Carmichaela, które je spełniają dla każdego $a \in \mathbb{Z}_n^*$.

Definicja 1.9 ([6]). *Liczbę naturalną złożoną n, która spełnia warunek $a^{n-1} \equiv 1 \ (mod\ n)$ dla każdej liczby naturalnej a takiej, że $NWD(a, n) = 1$, nazywamy liczbą Carmichaela.*

W rezultacie o liczbach Carmichaela przy podstawach a z nimi względnie pierwszymi algorytm zawsze orzeknie, że są pierwsze. Najmniejszą liczbą Carmichaela jest 561. Wśród n mniejszych od 100 000 000 jest zaledwie 255 liczb Carmichaela.

1.2.2 Test Millera-Rabina

Test Millera-Rabina to algorytm probabilistyczny, który ma największe zastosowanie ze wszystkich probabilistycznych testów pierwszości ze względu na wysokie prawdopodobieństwo poprawności określenia pierwszości danej liczby. Klasyfikuje się go jako jeden

z algorytmów Monte Carlo (nazwą tą określa się algorytmy probabilistyczne, których dane wyjściowe mogą być niepoprawne z konkretnym, zazwyczaj małym prawdopodobieństwem). Test Millera-Rabina opiera się na wniosku z małego twierdzenia Fermata, na poniższym twierdzeniu z arytmetyki modularnej oraz na odpowiednim wnioskowaniu opartym o wiedzę z teorii liczb.

Twierdzenie 1.10 ([6]). *Niech p będzie liczbą pierwszą. Wówczas*

$$x^2 \equiv 1 \ (mod\ p) \tag{1.3}$$

wtedy i tylko wtedy, gdy $x \equiv \pm 1 \ (mod\ p)$.

Dowód.

$$\begin{aligned} x^2 \equiv 1 \ (mod\ p) &\Leftrightarrow x^2 - 1 \equiv 0 \ (mod\ p) \Leftrightarrow (x-1)(x+1) \equiv 0 \ (mod\ p) \\ &\Leftrightarrow p|(x-1)(x+1) \Leftrightarrow x - 1 \equiv 0 \ (mod\ p) \vee x + 1 \equiv 0 \ (mod\ p) \\ &\Leftrightarrow x \equiv 1 \ (mod\ p) \vee x \equiv -1 \ (mod\ p) \end{aligned}$$

□

Definicja 1.11 ([6]). *Liczbę x nazywamy nietrywialnym pierwiastkiem kwadratowym modulo n z jedności, jeśli spełnia ona kongruencję $x^2 \equiv 1 \ (mod\ n)$ oraz $x \not\equiv \pm 1 \ (mod\ n)$.*

Przykład 1.12 ([6]). *Liczba 6 jest nietrywialnym pierwiastkiem kwadratowym modulo 35 z jedności, ponieważ $6^2 \equiv 1 \ (mod\ 35)$ i $6 \not\equiv \pm 1 \ (mod\ 35)$.*

Wniosek 1.13 ([6]). *Jeśli istnieje nietrywialny pierwiastek modulo n z jedności, to n jest liczbą złożoną.*

Lemat 1.14 ([8]). *Jeśli $p > 2$ jest nieparzystą liczbą pierwszą, to dla każdego $a \in \mathbb{Z}_p^*$ mamy*

$$a^m \equiv 1 \ (mod\ p) \tag{1.4}$$

albo

$$a^{m \cdot 2^i} \equiv -1 \ (mod\ p) \ dla\ pewnego \ \ 0 \leqslant i < s \ , \tag{1.5}$$

gdzie m–nieparzyste i s to liczby z zapisu $p - 1$ w postaci

$$p - 1 = m \cdot 2^s \ . \tag{1.6}$$

Dowód. Skoro p jest liczbą pierwszą, to dla każdego $a \in \mathbb{Z}_p^*$ z równości (1.6) oraz z wniosku 1.3 otrzymujemy, że

$$a^{m \cdot 2^s} = a^{p-1} \equiv 1 \ (mod\ p)\ . \tag{1.7}$$

Z twierdzenia 1.10 wynika, że element $x = a^{m \cdot 2^{s-1}} \equiv \pm 1 \ (mod\ p)$. Warto zauważyć, że jest to pierwiastek kwadratowy z liczby $a^{m \cdot 2^s}$ mod p. Jako że w następującym ciągu

$$\{a^m,\ a^{m \cdot 2},\ a^{m \cdot 2^2},\ ...,\ a^{m \cdot 2^{s-1}},\ a^{m \cdot 2^s}\}\ mod\ p$$

kolejne jego wyrazy są tak naprawdę poprzednimi wyrazami podniesionymi do kwadratu i ostatni wyraz przystaje do jedności (co zaobserowaliśmy w linijce (1.7)), to z twierdzenia 1.10 wnioskujemy, że albo wszystkie wyrazy poprzedzające ostatni muszą przystawać do 1 (w tym wyraz $a^m\ mod\ p$), albo, otrzymując kolejne pierwiastki kwadratowe (pierwiastkując najpierw ostatni wyraz, potem przedostatni itd.)), natrafimy na wyraz przystający do -1. Zatem zachodzi (1.4) albo (1.5).

□

Uwaga 1.15. *Stwierdzenie odwrotne do lematu 1.14 nie jest w ogólności prawdziwe.*

Dowód. Aby to pokazać, wystarczy znaleźć liczbę a względnie pierwszą z liczbą złożoną n, dla której zachodzi albo warunek (1.4), albo (1.5). Weźmy $n = 121 = 11 \cdot 11$ i $a = 3$, które są względnie pierwsze. Liczbę 121 możemy zapisać w postaci

$$n - 1 = 121 - 1 = 15 \cdot 2^3.$$

Stąd otrzymujemy, że $m = 15$, a $s = 3$. Obliczamy a^m mod n, podstawiając odpowiednie liczby a, n i m:

$$a^m = 3^{15} \equiv 1 \ (mod\ 121).$$

Widzimy, że dla tak zadanych liczb warunek (1.4) jest spełniony. Zatem pokazaliśmy, że z tego, że n spełnia jeden z warunków lematu 1.14, nie wynika, że n jest zawsze pierwsza.

□

W poniższym algorytmie k oznacza liczbę powtórzeń głównej jego części i im większa zostanie podana jej wartość, tym większa będzie szansa na znalezienie potencjalnego świadka złożoności dla n (definicja 1.17) za cenę wydłużenia jego pracy. W algorytmie liczby parzyste nie są rozpatrywane, ponieważ nie są pierwsze.

Algorytm 3 Test Millera-Rabina

- Podajemy liczbę $n > 2$, której pierwszość chcemy sprawdzić. Zakładamy, że n jest nieparzyste, wtedy możemy $n-1$ przedstawić w postaci:

$$n - 1 = m \cdot 2^s \ , \tag{1.8}$$

  gdzie m jest nieparzyste.

- Podajemy liczbę k oznaczającą maksymalną ilość powtórzeń kolejnych kroków.

- Powtarzamy poniższe procedury k razy, o ile algorytm wcześniej nie stwierdzi, że n jest złożona albo pierwsza.

  — Losujemy liczbę naturalną $1 < a < n$.

  — Sprawdzamy, czy $NWD(a, n) = 1$. Jeśli $NWD(a, n) > 1$, to n jest złożona i można zakończyć algorytm.

  — Obliczamy a^m mod n. Jeżeli

$$a^m \equiv 1 \ (mod\ n) \ , \tag{1.9}$$

  to n jest pierwsza i można zakończyć algorytm.

  — Jeżeli

$$a^m \not\equiv 1 \ (mod\ n) \ , \tag{1.10}$$

  to obliczamy kolejno

$$a^{m \cdot 2}\ mod\ n,\ a^{m \cdot 2^2}\ mod\ n,\ ...,\ a^{m \cdot 2^s}\ mod\ n\ . \tag{1.11}$$

  Jeśli wśród tych liczb jest jedynka, np. $a^{m \cdot 2^i} \equiv 1 \ (mod\ n)$, to patrzymy na poprzedni element

$$x = a^{m \cdot 2^{i-1}}\ mod\ n\ . \tag{1.12}$$

  Jeżeli $x \neq -1$, to n jest złożona i można zakończyć algorytm.

  — Jeśli w tym ciągu liczb nie ma jedynki, to n jest złożona i można zakończyć algorytm.

- Jeżeli po każdym z k powtórzeń dla elementu (1.12) zachodzi $x = -1$, to algorytm orzeka, że n jest pierwsza.

Poniżej przedstawiono przykład przebiegu testu Millera-Rabina dla liczby 41.

Przykład 1.16. *Prześledźmy algorytm Millera-Rabina dla liczby nieparzystej* $n = 41$.

Możemy ją zapisać w postaci:

$$n - 1 = 41 - 1 = 5 \cdot 2^3.$$

Stąd $m = 5$ *i* $s = 3$. *Załóżmy, że chcemy sprawdzić pierwszość liczby* 41 *w maksymalnie dwóch powtórzeniach. Rozpatrzmy pierwszą iterację* ($k = 1$)*: losujemy liczbę* $1 < a < 41$. *Wylosowano* $a = 5$, $NWD(5, 41) = 1$. *Obliczamy* a^m *mod* n*:*

$$a^m = 5^5 \equiv 9 \not\equiv 1 \ (mod\ 41)\ .$$

Warunek (1.9) *nie jest spełniony, zatem szukamy dla kolejnych wartości* s *wyrazu ciągu* (1.11)*, który będzie przystawał do* 1*:*

$$a^{m \cdot 2} = 5^{5 \cdot 2} \equiv -1 \ (mod\ 41)\ ,$$

$$a^{m \cdot 2^2} = 5^{5 \cdot 2^2} \equiv 1 \ (mod\ 41)\ .$$

Element ciągu (1.11) *przy* $s = 2$ *przystaje do* 1*, zatem patrzymy na element go poprzedzający, czyli przy* $s = 1$. *Przystaje on do* -1*, zatem przechodzimy do drugiej iteracji* ($k = 2$)*: losujemy liczbę* $a = 6$, $NWD(6, 41) = 1$. *Obliczamy* a^m *mod* n*:*

$$a^m = 6^5 \equiv 27 \not\equiv 1 \ (mod\ 41)\ .$$

Warunek (1.9) *nie jest spełniony, więc ponownie szukamy jedynki w ciągu* (1.11)*:*

$$a^{m \cdot 2} = 6^{5 \cdot 2} \equiv 32 \ (mod\ 41)$$

$$a^{m \cdot 2^2} = 6^{5 \cdot 2^2} \equiv -1 \ (mod\ 41)$$

$$a^{m \cdot 2^3} = 6^{5 \cdot 2^3} \equiv 1 \ (mod\ 41)$$

Element ciągu (1.11) *przy* $s = 3$ *przystaje do* 1*, a element go poprzedzający przystaje do* -1 (*tak jak w poprzedniej iteracji*)*. Dla tak zadanych na wstępie liczb algorytm zwróciłby informację, że* 41 *jest pierwsza.*

Definicja 1.17. *W teście Millera-Rabina taką liczbę* a*, dla której* $NWD(a, n) = 1$, $a^m \not\equiv 1$ ($mod\ n$) *oraz* $a^{m \cdot 2^i} \not\equiv -1$ ($mod\ n$) *dla każdego* $0 < i < s$ (*przy czym* $n = m \cdot 2^s + 1$ *jest nieparzystą liczbą złożoną, gdzie* m *jest nieparzyste*)*, będziemy nazywać świadkiem złożoności dla* n*, ponieważ zaświadcza ona, że* n *jest złożona.*

W algorytmie Millera-Rabina jeśli n jest liczbą pierwszą, to zachodzi albo (1.4), albo (1.5) dla każdego $a \in \mathbb{Z}_n^*$ (co wynika z lematu 1.14), stąd algorytm zawsze zwróci dla niej poprawną informację, że jest pierwsza (tak jak w przykładzie 1.16). Zatem jeśli znajdziemy jakiekolwiek $a \in \mathbb{Z}_n^*$, takie że n nie spełnia żadnego z tych dwóch warunków, to n na pewno jest liczbą złożoną.

Uwaga 1.18. *Stwierdzenie, że jeśli w ciągu* (1.11) *nie ma jedynki, to n jest złożona, wynika z wniosku 1.3. Wtedy zachodzi*

$$a^{m \cdot 2^s} = a^{n-1} \not\equiv 1 \ (mod\ n) \ .$$

Uwaga 1.19. *Stwierdzenie w teście Millera-Rabina, że jeśli wyraz* (1.12) *jest różny od* -1*, to n jest złożona, wynika z wniosku 1.13.*

W przypadku gdy zostanie wylosowana liczba $a \in \mathbb{Z}_n^*$ i nie będzie ona świadkiem złożoności dla n (czyli gdy zachodzi (1.9), albo wyraz (1.12) przystaje do -1), algorytm może się pomylić, orzekając o liczbie złożonej n, że jest pierwsza. Element prawdopodobieństwa w teście Millera-Rabina wynika z istnienia liczb silnie pseudopierwszych przy podstawie a.

Definicja 1.20. *Nieparzystą liczbę złożoną $n = m \cdot 2^s + 1$, gdzie m jest nieparzyste, nazywamy liczbą silnie pseudopierwszą przy podstawie a, kiedy zachodzi $a^m \equiv 1 \ (mod\ n)$ albo $a^{m \cdot 2^i} \equiv -1 \ (mod\ n)$ dla pewnego $0 \leqslant i < s$.*

Przykładem takiej liczby jest liczba złożona $n = 35 = 5 \cdot 7$, która jest silnie pseudopierwsza przy podstawie 34. Po jednokrotnym przeprowadzeniu dla niej algorytmu Millera-Rabina dla wylosowanej liczby $a = 34$ otrzymano by informację, że liczba 35 jest pierwsza, co jest błędnym stwierdzeniem. W tym przypadku kolejne przeprowadzenie algorytmu dla innej podstawy, np. $a = 6$ zwróciłoby poprawną informację (co zostało przedstawione w przykładzie 1.21).

Przykład 1.21. *Chcemy sprawdzić pierwszość liczby $n = 35$. Możemy ją zapisać w postaci*

$$n - 1 = 35 - 1 = 17 \cdot 2^1.$$

Stąd $m = 17$ i $s = 1$. Niech $k = 3$. Spójrzmy na pierwszą iterację ($k = 1$): losujemy liczbę $1 < a < 35$. Wylosowano $a = 34$, $NWD(34, 35) = 1$. Obliczamy a^m mod n:

$$a^m = 34^{17} \equiv -1 \not\equiv 1 \ (mod\ 35) \ .$$

Warunek (1.9) *nie jest spełniony, zatem szukamy dla kolejnych wartości s wyrazu ciągu* (1.11)*, który będzie przystawał do* 1*:*

$$a^{m \cdot 2} = 34^{17 \cdot 2} \equiv 1 \ (mod\ 35) \ .$$

Element ciągu (1.11) *przy $s = 1$ przystaje do* 1*, a element go poprzedzający (a^m mod* 35*) przystaje do -1, zatem przechodzimy do drugiej iteracji ($k = 2$). Losujemy liczbę $a = 6$, $NWD(6, 35) = 1$. Obliczamy a^m mod n:*

$$a^m = 6^{17} \equiv 6 \not\equiv 1 \ (mod\ 35) \ .$$

Warunek (1.9) *nie jest spełniony, więc ponownie szukamy jedynki w ciągu* (1.11)*:*

$$a^{m \cdot 2} = 6^{17 \cdot 2} \equiv 1 \ (mod\ 35)\ .$$

Element ciągu (1.11) *przy* $s = 1$ *przystaje do* 1*, a element go poprzedzający* $x = 6 \not\equiv -1$ $(mod\, 35)$*, więc znaleźliśmy liczbę* $a = 6$*, która jest nietrywialnym pierwiastkiem z* 1 *modulo* 35 *i jednocześnie świadkiem złożoności dla* 35*, więc algorytm orzeka, że* 35 *jest złożona (koniec algorytmu).*

Jeśli test Millera-Rabina zwróci informację, że liczba n jest pierwsza, to po przeprowadzeniu k powtórzeń prawdopodobieństwo tego, że jest to prawda, wynosi co najmniej $1 - \frac{1}{4^k}$ (jeśli n jest liczbą złożoną, to co najmniej $\frac{3}{4}$ wszystkich a jest dla niej świadkami złożoności). Dowód tego stwierdzenia można znaleźć m.in. pod pozycją nr [6].

Rozdział 2

Generowanie liczb pierwszych

W celu wygenerowania wszystkich liczb pierwszych z danego przedziału korzysta się z tak zwanych sit. Są to algorytmy, które, mając wygenerowaną listę wszystkich liczb naturalnych aż do ustalonej liczby, stopniowo usuwają z niej liczby złożone lub wykonują na niej inne operacje, które mają na celu zwrócenie listy tylko z liczbami pierwszymi.

Wygenerowana przez sito pula liczb pierwszych jest często wykorzystywana w testach pierwszości, aby wstępnie sprawdzić podzielność liczby n przez pewną ilość małych liczb pierwszych, co może zapobiec niepotrzebnym dodatkowym obliczeniom. W dalszej części rozdziału przedstawiono trzy wybrane sita, które w ostatnim podrozdziale zostały porównane ze sobą ze względu na zależność między długością przedziału a czasem ich działania w oparciu o napisany do tego program.

2.1 Sito Eratostenesa

Algorytm 4 Sito Eratostenesa

- Podajemy liczbę naturalną $n > 1$, która określa górny kraniec przedziału poszukiwania liczb pierwszych.
- Tworzymy listę wszystkich liczb naturalnych z przedziału $[2, n]$.
- Na początek niech $p = 2$ będzie najmniejszą liczbą pierwszą. Powtarzamy poniższe procedury dopóki $p \leqslant \lfloor \sqrt{n} \rfloor$.
  - Z listy usuwamy wszystkie wielokrotności liczby p o postaci pm, gdzie $p < pm \leqslant n$, o ile nie zostały już wcześniej usunięte.
  - Bierzemy kolejną najmniejszą pozostałą liczbę na liście i oznaczamy ją jako p.
- Nieusunięte liczby tworzą listę wszystkich liczb pierwszych z przedziału $[2, n]$.

Poniżej przedstawiono przykład wygenerowania liczb pierwszych z przedziału $[2, 50]$ sitema Eratostenesa.

Przykład 2.1. *Prześledźmy algorytm sita Eratostenesa dla* $n = 50$. *Tworzymy listę wszystkich liczb całkowitych od* 2 *do* 50 *i wykreślamy z niej liczby postaci:* $2 < 2m \leqslant 50$ (*gdzie* $m = 2, 3, ..., 25$), $3 < 3m \leqslant 50$ (*gdzie* $m = 2, 3, .., 16$), ... , $10 < 10m \leqslant 50$ (*gdzie* $m = 2, 3, ..., 5$).

	2	3	~~4~~	5	~~6~~	7	~~8~~	~~9~~	~~10~~	11	~~12~~	13	~~14~~	~~15~~	~~16~~	17	~~18~~
19	~~20~~	~~21~~	~~22~~	23	~~24~~	~~25~~	~~26~~	~~27~~	~~28~~	29	~~30~~	31	~~32~~	~~33~~	~~34~~	~~35~~	~~36~~
37	~~38~~	~~39~~	~~40~~	41	~~42~~	43	~~44~~	~~45~~	~~46~~	~~47~~	~~48~~	~~49~~	~~50~~				

Tym sposobem wygenerowaliśmy następującą listę liczb pierwszych

$$2 \quad 3 \quad 5 \quad 7 \quad 11 \quad 13 \quad 17 \quad 19 \quad 23 \quad 29 \quad 31 \quad 37 \quad 41 \quad 43\ .$$

W algorytmie sita Eratostenesa można przyjąć (tak jak to zrobiono), że wystarczy powtarzać procedury jedynie dla $p \leqslant \sqrt{n}$ zamiast dla $p \leqslant n$. Aby zrozumieć czemu, rozpatrzono graniczny przypadek. Jeśli $p < \sqrt{n}$, to, tak jak zostało pokazane w algorytmie, usuwa się wielokrotności p. Dla liczby $p = \sqrt{n}$ jej wielokrotności są następującej postaci:

$$\sqrt{n} \cdot 2\ ,\ \sqrt{n} \cdot 3\ ,\ \sqrt{n} \cdot 4\ , ...\ ,\ \sqrt{n} \cdot \sqrt{n} = n\ .$$

Można zauważyć, że w algorytmie wystarczy brać pod uwagę jedynie wielokrotności p większe lub równe p^2, ponieważ wszystkie mniejsze wielokrotności zostają wcześniej usunięte podczas wykreślania wielokrotności kolejno dwójki ($2 \cdot 2$, ... , $2 \cdot \sqrt{n}$, ... , $4 \cdot \sqrt{n}$, ...), trójki ($3 \cdot 2$, ... , $3 \cdot \sqrt{n}$, ...), piątki ($5 \cdot 2$, ... , $5 \cdot \sqrt{n}$, ...), ... itd. Natomiast dla $p = \sqrt{n} + 1$, wiedząc powyższe, wiadomo, że wielokrotności mniejszych od p^2 już nie ma i została ostatnia postaci $p^2 = (\sqrt{n} + 1)(\sqrt{n} + 1) > n$, która wychodzi poza ustalony przedział. To samo dzieje się dla pozostałych $\sqrt{n} < p \leqslant n$.

2.2 Sito Sundarama

Przy opracowywaniu tego podrozdziału korzystano z [10].

Algorytm 5 Sito Sundarama

- Podajemy liczbę naturalną $n > 1$, która wyznacza koniec przedziału, z którego chcemy wygenerować wszystkie nieparzyste liczby pierwsze poniżej liczby $2n + 2$.
- Tworzymy listę wszystkich liczb naturalnych z przedziału $[1, n]$.
- Usuwamy z tej listy wszystkie liczby postaci $i + j + 2ij$, gdzie:
  - — $i, j \in \mathbb{N}, 1 \leqslant i \leqslant j$,
  - — $i + j + 2ij \leqslant n$.
- Pozostałe liczby podwajamy i powiększamy o 1, tworząc w ten sposób listę wszystkich liczb pierwszych z przedziału $[3, 2n + 1]$.

Poniżej przedstawiono przykład wygenerowania liczb pierwszych z przedziału $[3, 101]$ sitema Sundarama.

Przykład 2.2. *Prześledźmy algorytm sita Sundarama dla $n = 50$. Tworzymy listę wszystkich liczb całkowitych od 1 do 50 i wykreślamy z niej liczby postaci $i + j + 2ij \leqslant 50$ z określonymi w algorytmie warunkami:*

$$1 + 1 + 2 \cdot 1 \cdot 1 = 4$$

$$1 + 2 + 2 \cdot 1 \cdot 2 = 7$$

$$1 + 3 + 2 \cdot 1 \cdot 3 = 10$$

$$2 + 2 + 2 \cdot 2 \cdot 2 = 12$$

$$\ldots$$

1	2	3	4	5	6	~~7~~	8	9	~~10~~	11	~~12~~	~~13~~	14	15	~~16~~	~~17~~	18
~~19~~	20	21	~~22~~	23	~~24~~	~~25~~	26	~~27~~	~~28~~	29	30	~~31~~	~~32~~	33	~~34~~	35	36
~~37~~	~~38~~	39	~~40~~	41	~~42~~	~~43~~	44	~~45~~	~~46~~	~~47~~	48	~~49~~	50 .				

Pozostałe liczby podwajamy i powiększamy o 1, generując tym samym następującą listę liczb pierwszych

3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
61 67 71 73 79 83 89 97 101 .

Powyższy algorytm sprowadza się do utworzenia nowej listy, w której znajdują się liczby nieparzyste większe od 2, które nie są złożone. Wygenerowanie liczb nieparzystych zostaje osiągnięte poprzez utworzenie listy liczb postaci $2k+1$ (podwojenie i powiększenie o 1), gdzie k to liczby z początkowego przedziału $[1, n]$. Pierwszość liczb nieparzystych z tej listy została zagwarantowana w momencie usunięcia z początkowego przedziału liczb postaci $k = i+j+2ij$ (z zadanymi w algorytmie warunkami), ponieważ to właśnie te liczby okazują się być złożone po podwojeniu i powiększeniu o 1. Rzeczywiście, podstawiając za k liczbę $i + j + 2ij$, otrzymuje się, że

$$m = 2(i + j + 2ij) + 1 = 2i + 2j + 4ij + 1 = (2i + 1)(2j + 1) \ .$$

Stąd widać, że nieparzysta liczba m o takiej postaci jest liczbą złożoną.

2.3 Sito Atkina

Przy opracowywaniu tego podrozdziału korzystano m.in. z [9].

Algorytm 6 Sito Atkina

- Podajemy liczbę naturalną $n > 5$, która określa górny kraniec przedziału poszukiwania liczb pierwszych.

- Liczby $2, 3, 5$ wpisujemy do listy liczb pierwszych.

- Tworzymy listę sita wszystkich liczb naturalnych z przedziału $[6, n]$ i domyślnie przypisujemy każdej z nich oznaczenie "złożona".

- Dla każdej kolejnej liczby $6 \leqslant k \leqslant n$ obliczamy k mod 60. Wynik oznaczamy przez r.

  — Jeśli r jest równe $1, 13, 17, 29, 37, 41, 49$ lub 53 (mod 60) oraz jeśli liczba rozwiązań równania
  $$4x^2 + y^2 = k \tag{2.1}$$
  jest nieparzysta dla $x, y \in \mathbb{N}_+$, to zmieniamy oznaczenie liczby k na liście sita na "pierwsza".

  — Jeśli r jest równe $7, 19, 31$ lub 43 (mod 60) oraz jeśli liczba rozwiązań równania
  $$3x^2 + y^2 = k \tag{2.2}$$
  jest nieparzysta dla $x, y \in \mathbb{N}_+$, to zmieniamy oznaczenie liczby k na liście sita na "pierwsza".

  — Jeśli r jest równe $11, 23, 47$ lub 59 (mod 60) oraz jeśli liczba rozwiązań równania
  $$3x^2 - y^2 = k \tag{2.3}$$
  jest nieparzysta dla $x, y \in \mathbb{N}_+$, gdzie $x > y$, to zmieniamy oznaczenie liczby k na liście sita na "pierwsza".

  — Jeśli r nie równa się żadnej z powyższych reszt, to nie zmieniamy oznaczenia k na liście sita.

- Z pozostałych na liście sita liczb oznaczonych jako "pierwsza" wybieramy najmniejszą z nich, oznaczamy ją jako p i umieszczamy na liście liczb pierwszych z przedziału $[2, n]$. Podnosimy p do kwadratu, a następnie liczbę p^2 oraz wszystkie jej wielokrotności postaci $p^2 i$, takie że
$$p^2 < p^2 i \leqslant n \ ,$$
oznaczamy na liście sita jako "złożone".

- Powyższy krok powtarzamy do momentu, kiedy nie znajdziemy już na liście sita liczby oznaczonej jako "pierwsza". Wtedy można zakończyć algorytm.

Poniżej przedstawiono przykład wygenerowania liczb pierwszych z przedziału $[2, 169]$ sitem Atkina.

Przykład 2.3. *Prześledźmy algorytm sita Atkina dla wybranych liczb przy* $n = 169$. *Umieszczamy w liście liczb pierwszych liczby* $2, 3, 5$. *Dla* $k = 6$ *obliczamy*

$$k \ mod \ 60 = 6 \ mod \ 60 = 6 = r \ .$$

$r = 6$ *nie jest żadną z wypisanych w algorytmie reszt, zatem do końca algorytmu liczba* 6 *pozostaje oznaczona jako "złożona". Weźmy kolejną liczbę* $k = 7$. *Obliczamy*

$$k \ mod \ 60 = 7 \ mod \ 60 = 7 = r \ .$$

Sprawdzamy liczbę rozwiazań równania $3x^2 + y^2 = 7$. *Para liczb spełniająca to równanie, to*

$$(x, y) = (1, 2) \ .$$

Jest to jedyne rozwiązanie tego równania, zatem liczba rozwiązań równania (2.2) *jest nieparzysta, więc liczbę* 7 *oznaczamy jako "pierwszą". Po przeprowadzeniu algorytmu dla kolejnych liczb dochodzimy do ostatniej* $k = 169$. *Obliczamy*

$$k \ mod \ 60 = 169 \ mod \ 60 = 49 = r \ .$$

Sprawdzamy liczbę rozwiazań równania $4x^2 + y^2 = 169$. *Para liczb spełniająca to równanie, to*

$$(x, y) = (6, 5) \ .$$

Mamy tylko jedno rozwiązanie, zatem liczba rozwiązań równania (2.1) *jest nieparzysta, więc liczbę* 169 *oznaczamy jako "pierwszą".*

Najmniejszą liczbą oznaczoną jako "pierwsza" na liście sita jest 7. *Umieszczamy ją na liście liczb pierwszych. Oznaczamy jako złożone liczby* $49, 98, 147$. *Następną liczbą pierwszą na liście sita jest* $p = 11$. *Umieszczamy ją na liście liczb pierwszych. Jako złożoną oznaczamy liczbę* $p^2 = 121$. *Kolejną liczbą pierwszą na liście sita jest* $p = 13$. *Umieszczamy ją na liście liczb pierwszych. Jest to ostatnia liczba, którą podnosimy do kwadratu, ponieważ*

$$p^2 = 13^2 = 169 = n \ .$$

Liczbę 169 *oznaczamy jako złożoną.*

Stąd, generując liczby wyłącznie z listy liczb pierwszych, otrzymujemy

	2	3	5	7	11	13	17	19	23	29	31	37	41	43
47	53	59	61	67	71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167 .					

Analizując przykład 2.3, można zauważyć istotność wykluczania liczb, które dzielą się przez liczby kwadratowe (definicja 2.4) w sicie Atkina. Otóż gdyby tej czynności nie przeprowadzono, liczba 169 zostałaby uznana za liczbę pierwszą.

Definicja 2.4. *Liczba kwadratowa to liczba całkowita, która powstała poprzez podniesienie do kwadratu innej liczby całkowitej.*

Definicja 2.5. *Liczba bezkwadratowa to liczba całkowita, która nie dzieli się przez żadną liczbę kwadratową oprócz* 1.

Uwaga 2.6. *Jeśli liczba nie jest kwadratowa, nie oznacza to, że musi być koniecznie bezkwadratowa, np.* 8 *nie jest liczbą kwadratową, ale nie jest też liczbą bezkwadratową, ponieważ dzieli się przez* $4 = 2^2$.

Poniżej umieszczono definicje oraz główne twierdzenia, na których opiera się algorytm Atkina. Dowody twierdzeń można znaleźć w [1].

Twierdzenie 2.7 ([1])**.** *Niech n będzie liczbą bezkwadratową, gdzie $n \in 1 + 4\mathbb{Z}$. Wtedy n jest pierwsza wtedy i tylko wtedy, gdy $\#\{(x, y) : x > 0,\ y > 0,\ 4x^2 + y^2 = n\}$ jest nieparzysta.*

Twierdzenie 2.8 ([1])**.** *Niech n będzie liczbą bezkwadratową, gdzie $n \in 1 + 6\mathbb{Z}$. Wtedy n jest pierwsza wtedy i tylko wtedy, gdy $\#\{(x, y) : x > 0,\ y > 0,\ 3x^2 + y^2 = n\}$ jest nieparzysta.*

Twierdzenie 2.9 ([1])**.** *Niech n będzie liczbą bezkwadratową, gdzie $n \in 11 + 12\mathbb{Z}$. Wtedy n jest pierwsza wtedy i tylko wtedy, gdy $\#\{(x, y) : x > y > 0,\ 3x^2 - y^2 = n\}$ jest nieparzysta.*

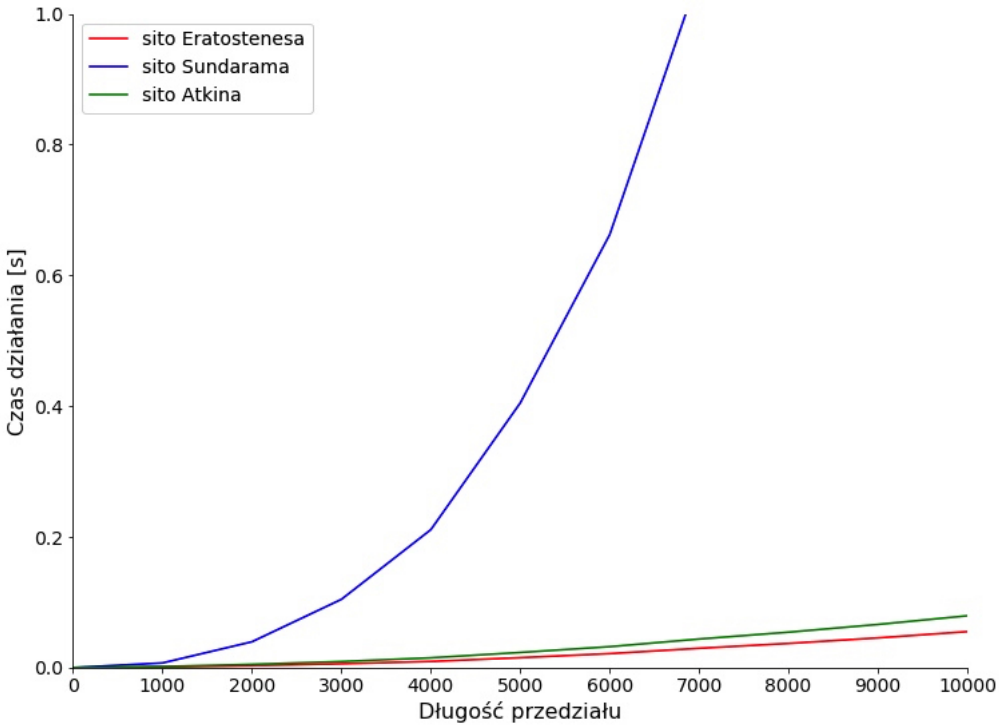
2.4 Porównanie przykładowo zaimplementowanych sit. Wybranie najszybszego z nich w zależności od długości zadanego przedziału

Przy opracowywaniu tego podrozdziału korzystano z [7].

W tym podrozdziale zostały ze sobą porównane trzy sita generujące liczby pierwsze przedstawione we wcześniejszej części rozdziału. Na potrzeby pracy każde z nich zostało zaimplementowane w przykładowy sposób (czasem bardziej zoptymalizowany niż ten przedstawiony w pracy), a następnie przy ich wykorzystaniu wygenerowano wykresy, na

których dla każdego z sit zestawiono ze sobą wybrane długości przedziałów, z których generują one liczby pierwsze, oraz odpowiadające im czasy działania. Następnie w jednej tabeli zestawiono czasy otrzymane podczas generowania wykresów dla poszczególnych sit przy wybranych długościach przedziałów w celu skonfrontowania ich z obserwacjami dokonanymi podczas analizy wykresów i wyciągnięcia odpowiednich wniosków.

Kod generujący jeden z takich wykresów wraz z funkcjami definiującymi algorytmy sit został umieszczony w dodatku A. Jest on w całości zaimplementowany w języku Python. Symulacja kodu wykonywana była w programie Jupyter Notebook.

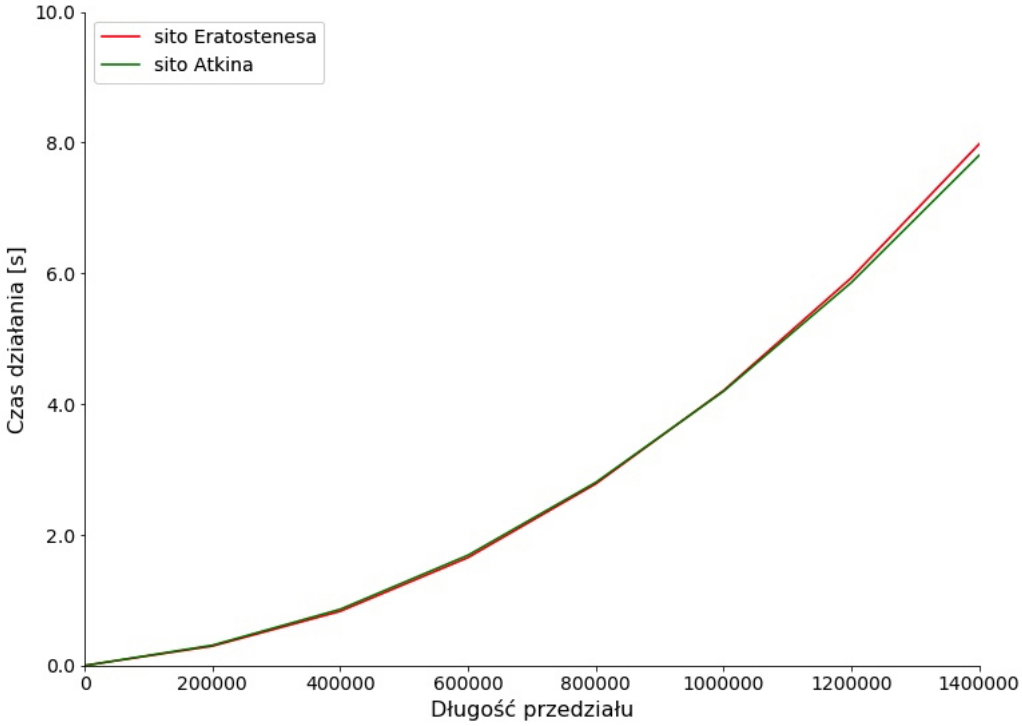


Rysunek 2.1: Czas działania poszczególnych sit dla przedziałów < 10 tys.

Na rys. 2.1 został przedstawiony wykres zależności czasu od długości przedziału (dla przedziałów mniejszych od 10 000) przy wywołaniu wszystkich trzech sit. Widać, że dla wygenerowanych przedziałów czas działania sita Eratostenesa oraz sita Atkina różni się nieznacznie i dla obu wynosi mniej niż 0.1 s dla przedziału długości 10 000, natomiast sito Sundarama ewidentnie działa najdłużej dla przedziałów większych niż 1 000, bo już np. ok. 1 s dla przedziału długości 7 000. Ze względu na ograniczenia czasowe nie będzie ono już brane pod uwagę w dalszej analizie sit dla większych przedziałów. Jego implementacja nie jest efektywna.

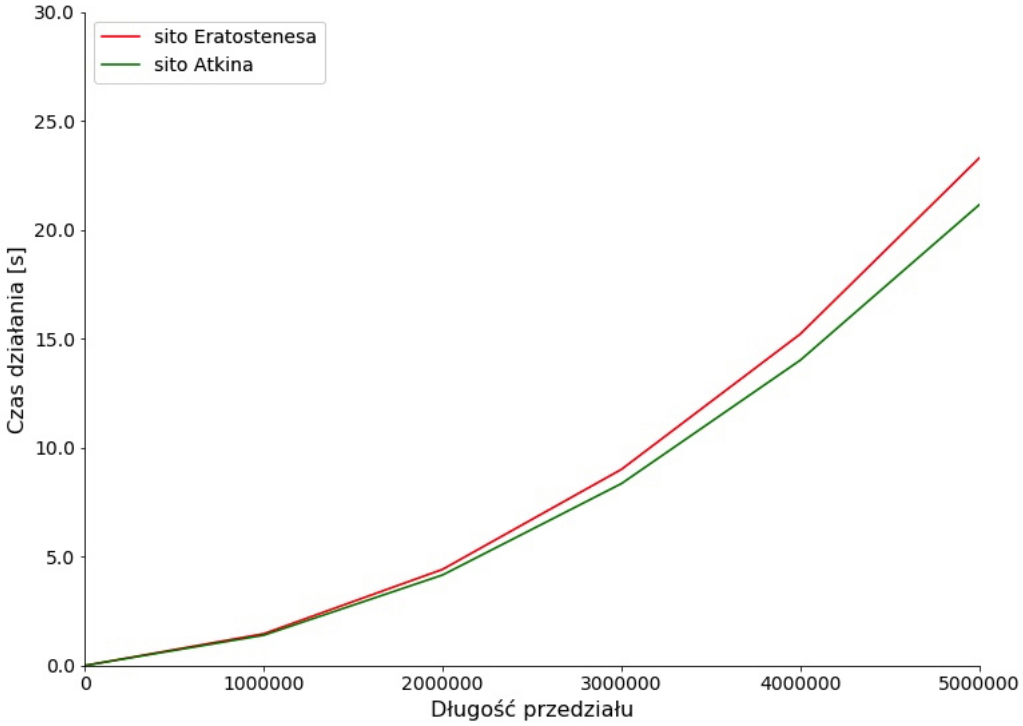
Na rys. 2.2 został przedstawiony wykres zależności czasu od długości przedziału (dla przedziałów mniejszych od 1 400 000) przy wywołaniu sit Eratostenesa i Atkina. Dla

przedziału równego 1 000 000 oba sita działają ok. 7 s. Można zauważyć minimalne różnice czasowe między nimi, ale to, który z nich jest na danych odcinkach szybszy, będzie lepiej widać w zestawieniu czasów. Aby zobaczyć, co dzieje się dla jeszcze większych przedziałów, wygenerowano kolejny wykres.



Rysunek 2.2: Czas działania poszczególnych sit dla przedziałów < 1.4 mln

Na rys. 2.3 został przedstawiony wykres zależności czasu od długości przedziału (dla przedziałów mniejszych od 5 000 000) przy wywołaniu sit Eratostenesa i Atkina. Widać na nim, że dla przedziałów powyżej wartości 1 500 000 sito Atkina jest najszybsze. Dla największej wartości na wykresie różnica między dwoma sitami wynosi ok. 2 s.



Rysunek 2.3: Czas działania poszczególnych sit dla przedziałów < 5 mln

Tabela 2.6: Czasy działania poszczególnych sit dla wybranych długości przedziałów

Długość przedziału	**s. Eratostenesa**	**s. Sundarama**	**s. Atkina**
100	0.000342	0.000453	0.000824
1 000	0.003787	0.022488	0.007229
10 000	0.055019	2.607875	0.079238
100 000	0.654818	-	0.768038
1 000 000	4.206323	-	4.195718
5 000 000	23.290262	-	21.147293

Dane w tabeli powyżej potwierdzają to, co zostało zauważone na wykresach. Dodatkowo można z niej wyczytać, że sito Eratostenesa jest najszybsze dla przedziałów w okolicach 1 000 000 oraz mniejszych od niego, a powyżej tej wartości to sito Atkina szybciej generuje liczby pierwsze. Poza tym widać, że nawet dla przedziałów krótszych niż 1 000 sito Sundarama nie jest szybsze od żadnego z pozostałych dwóch.

Ze względu na wielokrotne wywoływanie kodu, w celu uniknięcia wyświetlania zbędnych części kodu, podczas generowania wykresów w funkcjach definiujących sita zamieniono w kodzie linijkę

```
return print(lista_liczb_pierwszych)
```

odpowiedzialną za wyświetlanie listy liczb pierwszych na poniższą, która tego nie robi.

```
return lista_liczb_pierwszych
```

Powyższe modyfikacje oczywiście miały wpływ na uzyskane wyniki czasowe, ale nie miały wpływu na ustalenie, jakie sito jest najszybsze w zależności od długości zadanego przedziału.

Podczas wykonywania się kodu program przeprowadzał algorytm każdego sita 10-krotnie dla każdego z przedziałów pomiarowych i zwracał uśredniony wynik. Co prawda jest to mała ilość powtórzeń, ale, mając na uwadze wydłużający się czas działania programu dla coraz większych przedziałów, uznano, że jest wystarczająca do przeprowadzenia bardzo ogólnej analizy zaimplementowanych algorytmów.

Podsumowanie

W powyższej pracy zostały zaprezentowane dwa typy algorytmów związanych z liczbami pierwszymi.

W pierwszym rozdziale skupiono uwagę na dwóch probabilistycznych testach pierwszości. Na chwilę obecną to probabilistyczny algorytm pierwszości Millera-Rabina ma najszersze zastosowanie wśród testów pierwszości ze względu na czas w jakim działa oraz ze względu na wysokie prawdopodobieństwo jego poprawności. Jeśli czytelnika interesują również algorytmy istotne z teoretycznego punktu widzenia, to na pewno warto zainteresować się bardziej złożonymi testami deterministycznymi.

Drugi rozdział został poświęcony trzem sitom generującym liczby pierwsze. Każde z nich zostało przedstawione, a następnie na potrzeby pracy zaimplementowane w programie w przykładowy sposób, dzięki czemu możliwe było ich porównanie. Po przeanalizowaniu otrzymanych z ich zestawienia wyników sito Atkina okazało się być najszybsze dla długości przedziałów większych od ok. 1 000 000, a sito Eratostenesa najszybsze dla przedziałów od nich mniejszych. Implementację sita Sundarama można natomiast określić jako nieużyteczną przy dwóch pozostałych sitach. Należy pamiętać, że w żadnym razie załączone przykładowe implementacje sit nie są jedynymi i można szukać sposobów ich optymalizowania.

Jako następstwo ciągłego rozwoju techniki zrozumiałym jest szukanie nowych algorytmów, które będą jeszcze bardziej efektywne niż obecnie istniejące.

Wykaz literatury

[1] A. O. L. Atkin, D. J. Bernstein: *Prime sieves using binary quadratic forms.* Mathematics of Computation, Vol. 73, 2004, s. 1023-1030.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest: *Wprowadzenie do algorytmów.* Wydawnictwa Naukowo-Techniczne, Wyd. VII, Warszawa, 2005.

[3] M. O. Rabin: *Probabilistic algorithm for testing primality.* Journal of Number Theory, Vol. 12, 1980, s. 128-138.

[4] P. Ribenboim: *Mała księga wielkich liczb pierwszych.* Wydawnictwa Naukowo-Techniczne, Warszawa, 1997.

[5] A. Szepietowski: *Matematyka dyskretna.* Wydawnictwo Uniwersytetu Gdańskiego, Gdańsk, 2006.

[6] S. Y. Yan: *Teoria liczb w informatyce.* Wydawnictwo Naukowe PWN, Warszawa, 2006.

[7] http://compoasso.free.fr/primelistweb/page/prime/eratosthene_en.php [data dostępu: 18.06.2018].

[8] https://en.wikipedia.org/wiki/Miller–Rabin_primality_test
[data dostępu: 20.06.2018].

[9] https://en.wikipedia.org/wiki/Sieve_of_Atkin [data dostępu: 09.06.2018].

[10] https://en.wikipedia.org/wiki/Sieve_of_Sundaram [data dostępu: 09.06.2018].

[11] https://en.wikipedia.org/wiki/Strong_pseudoprime [data dostępu: 20.06.2018].

Dodatek A: Kod do rysowania wykresu porównującego sita

```
import math
import numpy as np
import time
import matplotlib.pyplot as plt
%matplotlib inline
from matplotlib.ticker import FormatStrFormatter


def sito_eratostenesa(n):
    """Funkcja zwracająca listę wszystkich liczb pierwszych aż do n
    zaimplementowana za pomocą algorytmu sita Eratostenesa."""

    lista_liczb_pierwszych = []
    sito = np.full((n), True)
    sito[0] = False

    for i in range(2, int(math.sqrt(n))+1):
        if sito[i-1] is not 0:
            j = i + i
            while(j <= n):
                sito[j-1] = False
                j += i

    for p in range(1, n+1):
        if sito[p-1]:
            lista_liczb_pierwszych.append(p)

    return lista_liczb_pierwszych   # print(lista_liczb_pierwszych)
```

```
def sito_sundarama(n):
    """Funkcja zwracająca listę wszystkich liczb pierwszych aż do n
    zaimplementowana za pomocą algorytmu sita Sundarama."""

    t = int(n/2)

    lista_liczb_pierwszych = [2]
    sito = np.full((t), True)

    for i in range(1, int(t/2)+1):
        for j in range(i, int(t/2)+1):
            m = i + j + 2*i*j
            if m <= t:
                sito[m-1] = False

    for p in range(1, t+1):
        if sito[p-1] and 2*p+1 <= n:
            lista_liczb_pierwszych.append(2*p+1)


    return lista_liczb_pierwszych   # print(lista_liczb_pierwszych)
```

```
def sito_atkina(n):
    """Funkcja zwracająca listę wszystkich liczb pierwszych aż do n
    zaimplementowana za pomocą algorytmu sita Atkina."""

    lista_liczb_pierwszych = [2, 3]
    sito = np.full((n), False)

    for x in range(1, int((math.sqrt(n))/(math.sqrt(4)))+1):
        for y in range(1, int(math.sqrt(n))+1):
            m = 4 * x**2 + y**2
            if m <= n and (m % 12 == 1 or m % 12 == 5):
                sito[m-1] = not sito[m-1]
    for x in range(1, int((math.sqrt(n))/(math.sqrt(3)))+1):
        for y in range(1, int(math.sqrt(n))+1):
            m = 3 * x**2 + y**2
            if m <= n and m % 12 == 7:
                sito[m-1] = not sito[m-1]
    for x in range(1, int((math.sqrt(n))/(math.sqrt(2)))+1):
        for y in range(1, int(math.sqrt(n))+1):
            if x > y:
                m = 3 * x**2 - y**2
                if m <= n and m % 12 == 11:
                    sito[m-1] = not sito[m-1]

    for k in range(5, int(math.sqrt(n))+1):
        if sito[k-1] is not 0:
            k2 = k * k
            for m in range(k2, n+1, k2):
                sito[m-1] = False

    for p in range(1, n+1):
        if sito[p-1]:
            lista_liczb_pierwszych.append(p)


    return lista_liczb_pierwszych   # print(lista_liczb_pierwszych)
```

```
przedzial_e, przedzial_s, przedzial_a = ([] for i in range(3))
czas_e, czas_s, czas_a = ([] for i in range(3))
total_e10, total_s10, total_a10 = (0 for i in range(3))

for n in range(3, 10005, 1000):
    for i in range(10):

        start = time.clock()
        sito_eratostenesa(n)
        end = time.clock()
        total_e = end-start
        total_e10 += total_e

        start = time.clock()
        sito_sundarama(n)
        end = time.clock()
        total_s = end-start
        total_s10 += total_s

        start = time.clock()
        sito_atkina(n)
        end = time.clock()
        total_a = end-start
        total_a10 += total_a

    przedzial_e.append(n)
    przedzial_s.append(n)
    przedzial_a.append(n)
    czas_e.append(total_e10/10)
    czas_s.append(total_s10/10)
    czas_a.append(total_a10/10)

print(czas_e, n)
print(czas_s, n)
print(czas_a, n)
```

```
plt.figure(figsize=(12, 9))
plt.plot(przedzial_e, czas_e, 'r', label='sito Eratostenesa')
plt.plot(przedzial_s, czas_s, 'b', label='sito Sundarama')
plt.plot(przedzial_a, czas_a, 'g', label='sito Atkina')
ax = plt.subplot(111)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.get_xaxis().tick_bottom()
ax.get_yaxis().tick_left()
plt.xlim(0, 10000)
plt.ylim(0, 1)
ax.yaxis.set_major_formatter(FormatStrFormatter('%.1f'))
plt.xticks(range(0, 10003, 1000), fontsize=14)
plt.yticks(fontsize=14)
plt.ylabel("Czas działania [s]", fontsize=16)
plt.xlabel("Długość przedziału", fontsize=16)
plt.legend(loc='upper left', fontsize=14)

plt.show()
```